

SYSTEM SPECIFICATION: COGNITIVE REPOSITORY & AGENT ORCHESTRATION

Document Target: Autonomous Planning, Auditing, and Execution Agents **Version:** 2.0
(Recursive P2P & Cross-Auditor Architecture) **Core Framework:** 5+1 Tier Cognitive Memory, Multi-Model Cross-Auditing, Sandboxed Red Gatekeeping, and Distributed RAG Manifests

1. Architectural Mission & Invariants

This specification governs the autonomous ingestion, skill/tool extraction, cross-model telemetry collection, adversarial red-team verification, and cloud-native RLVR (Reinforcement Learning via Verifiable Rewards) recursive script training across edge daily drivers, home P2P nodes, and cloud storage.

Non-Negotiable Invariants

- 1. **Sensory Immutability:** Content in 01_raw_sources/ is strictly write-once. No agent may modify original evidentiary source data.
- 2. **Dual Extraction Mandate:** Every document ingested into the ecosystem must be parsed simultaneously for **Procedural Skills** (prompt patterns, behavioral heuristics) and **Executable Tools** (scripts, wrappers, deterministic CLI invocations).
- 3. **Red Auditor Gatekeeping:** No episodic execution trace or generated script may enter the recursive training stream without passing an isolated sandbox evaluation by the **Red Auditor Agent**.
- 4. **Universal Source of Truth:** All local auditor nodes and the sandboxed Red Auditor must evaluate claims and script outcomes against the shared master memory bank and database.
- 5. **Distributed Manifest Coverage:** Every repository root and major structural subsystem must maintain its own local manifest.jsonl to power low-overhead hierarchical RAG retrieval.

2. Updated Repository Topology & Distributed Manifest Map

```
📦 vault_root/
├── 📄 MAP.md                                # Master human-readable
index & ontology graph
├── 📊 manifest.jsonl                        # Root-level RAG
registry & global hash table
|
├── 📁 01_raw_sources/                      # TIER 1: COLD SENSORY
```

ARCHIVE (Read-Only)	
—  manifest.jsonl	# Local RAG catalog of
all immutable raw sources	
—  pdf/	# Source whitepapers,
specs, data sheets	
—  media/	# Video captures, audio
memos, transcripts	
—  text/	# Raw text scrapes,
dumps, API payload exports	
—  02_master_wiki/	# TIER 2: SEMANTIC
MEMORY VAULT (Read/Write)	
—  manifest.jsonl	# Local RAG catalog of
conceptual nodes & graph links	
—  concepts/	# Atomic linked markdown
notes	
—  architectures/	# Blueprints, schema
designs, system specs	
—  entities/	# Domain registries,
data contracts, entity tables	
—  03_recall_cache/	# TIER 3: WORKING MEMORY
ACCELERATOR (Rebuilt/Ephemeral)	
—  manifest.jsonl	# Local RAG catalog of
vector & chunk shards	
—  jsonl/	# Pre-tokenized passages
for high-speed injection	
—  vectors/	# Dense vector indices
(FAISS/Chroma/HNSW checkpoints)	
—  kv_store/	# Low-latency key-value
indices	
—  04_skills_runtime/	# TIER 4: PROCEDURAL
REPertoire & EXTRACTED TOOLS	
—  manifest.jsonl	# Local RAG catalog of
all extracted skills & tools	
—  prompt_skills/	# Modular SKILL.md
prompt definitions (Extracted & Authored)	
—  extracted_tools/	# Deterministic tools
mined from ingested documentation	
— cli/	# Extracted command-line
utilities	
— wrappers/	# Extracted API
interfaces & micro-functions	
—  runtimes/	# Master operational
scripts and execution hooks	
—  policies/	# Self-correction
guidelines and validation schemas	

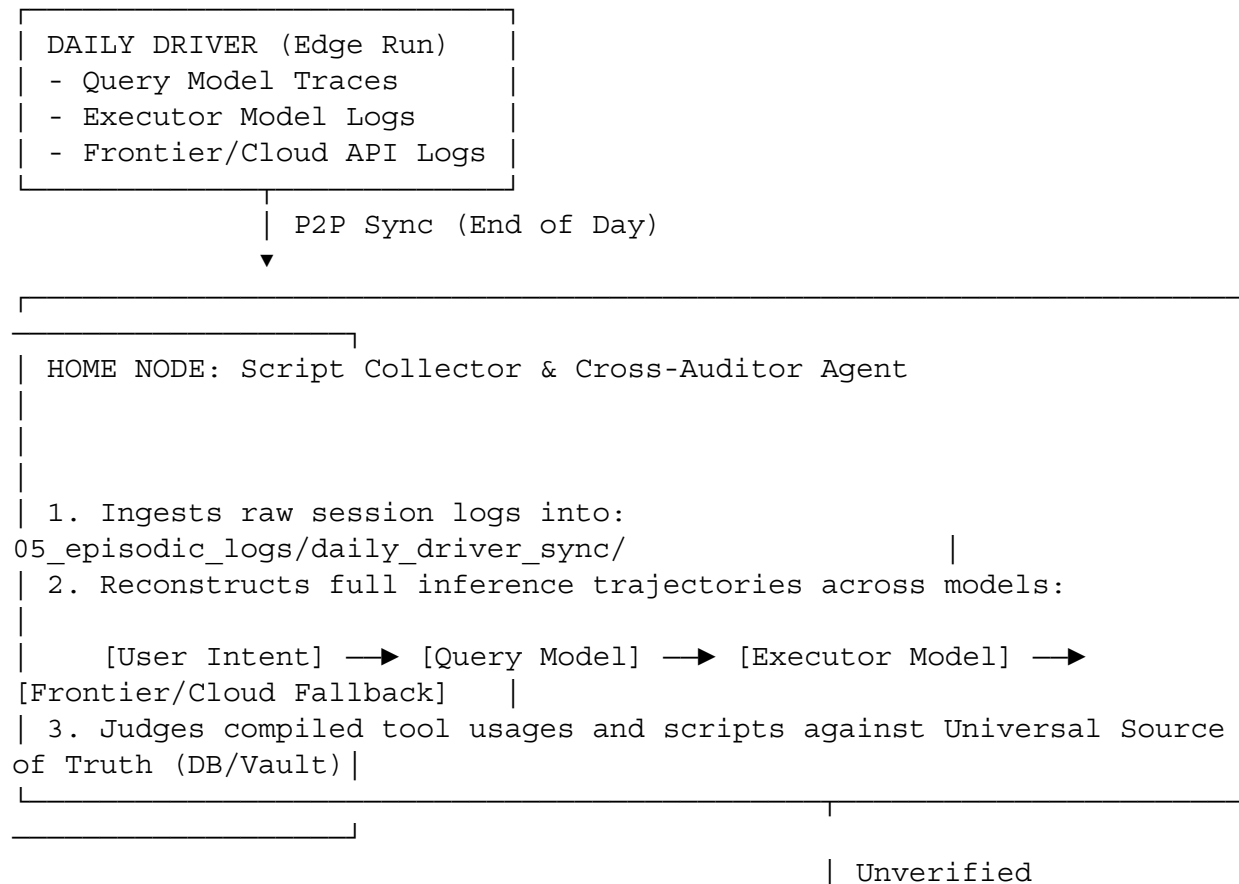
```

└─ 📁 05_episodic_logs/                                # TIER 5: TELEMETRY,
AUDITS & TRAJECTORIES                                   # Local RAG catalog of
  └─ 📊 manifest.jsonl                                   # Ingested logs from
sessions, audits, and verifiers                         # Unified multi-model
  └─ 📱 daily_driver_sync/                               # Red Auditor evaluation
edge device sessions                                   # Graded rewards,
  └─ ⌚ trajectories/                                    # Schema integrity
traces (Query, Executor, Frontier)
  └─ 🛑 red_audit_sandbox/
reports, pass/fail quarantine
  └─ ⚖️ rlvr_verifiers/
assertion outcomes (+1.0 / -1.0)
  └─ 🧹 hygiene_reports/
audits, broken link checks

```

3. End-of-Day P2P Synchronization & Cross-Auditor Topology

When the daily driver reconnects to the home environment via the peer-to-peer server mesh, the **Home Script Collector / Auditor Agent** takes over orchestration:



Trajectories

HOME NODE: Sandboxed Red Auditor Gatekeeper		
- Operates in an isolated execution sandbox.		
- Queries shared database / memory bank for independent truth verification.		
- Performs adversarial compliance, error boundary, and anti-hallucination checks.		
	PASS	FAIL
(-1.0)		
Quarantined Package for RLVR Cloud		
05_episodic_logs/		
Upload to GCS Bucket		
4 / Policy Update		
Quarantine Log:		
Ingest back into Tier		

Operational Steps of the Cross-Auditor

1. **Tri-Model Trace Alignment:** Chronologically interleaves prompts, tool calls, and outputs generated by:
 - o **Query Model:** Planning, prompt formulation, and intent decomposition.
 - o **Executor Model:** Local deterministic code runs and parameter parsing.
 - o **Frontier Model:** Complex edge-case resolution, advanced code synthesis, or fallbacks.
2. **Deterministic Script Compilation:** Extracts raw commands and code blocks from execution traces and packages them as standalone testable scripts.
3. **Database Grounding Check:** Cross-references all outputs against the master database and 02_master_wiki/ to ensure consistency with existing state.

4. Sandboxed Red Auditor Gatekeeping Protocol

Before any compiled script or trajectory payload is marked for recursive training, it must be routed through the **Red Auditor Agent**.

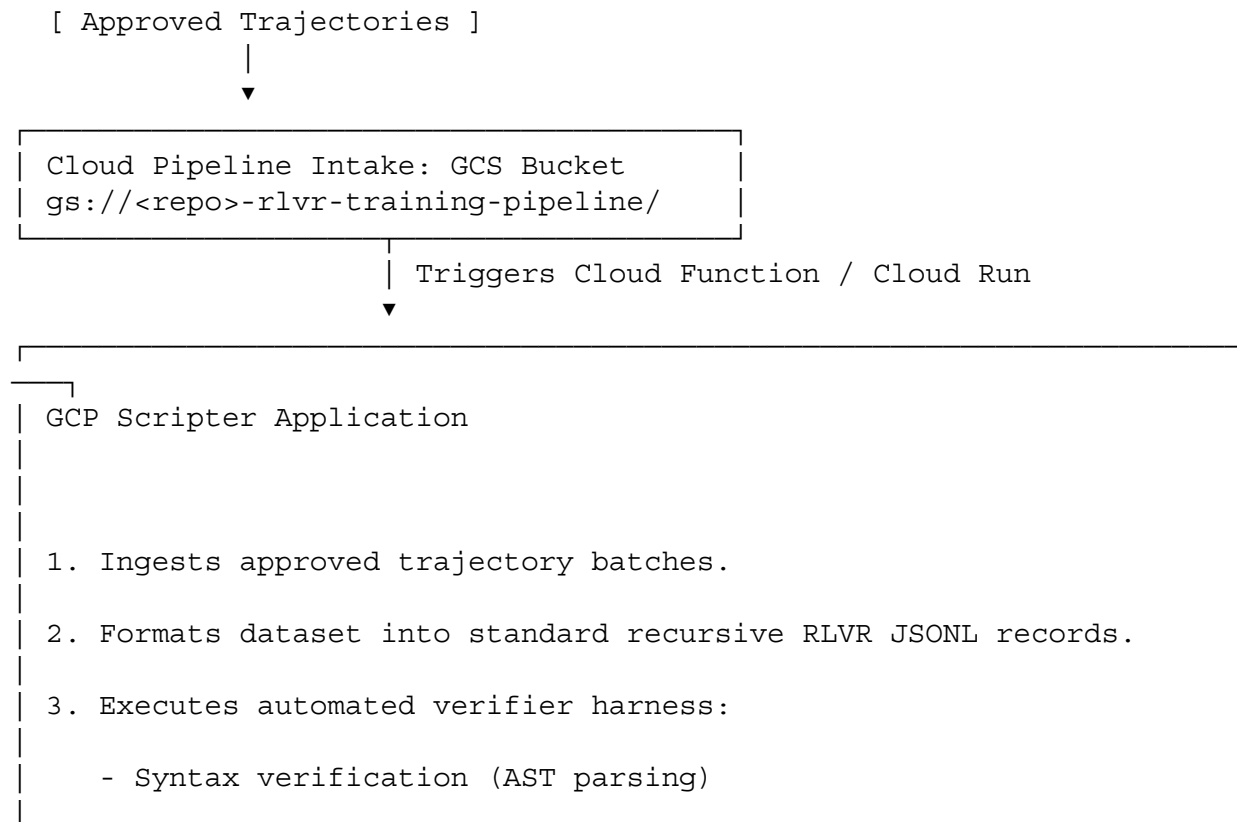
Sandbox Execution Constraints

- **Isolation:** Evaluates artifacts in an isolated container without network access.
- **Grounding Authority:** References the local master database and manifest.jsonl registries as its sole reference for ground truth.
- **Evaluation Criteria:**
 1. **Determinism:** Does the extracted script run cleanly without unhandled exceptions or hidden environment assumptions?
 2. **Truth Verification:** Did the model invent facts, falsify tool return schemas, or ignore state updates from the universal database?
 3. **Safety & Hygiene:** Does the script expose credentials, execute unsafe recursive deletions, or write outside permitted directories?

Disposition States

- **VERDICT_APPROVED (+1.0):** Output emitted to 05_episodic_logs/trajectories/approved_YYYYMMDD_batch.jsonl and prepared for cloud upload.
- **VERDICT_REJECTED (-1.0):** Output emitted to 05_episodic_logs/red_audit_sandbox/rejected_YYYYMMDD_batch.jsonl with an explicit failure reason, triggering recursive self-correction in Tier 4 policies.

5. Cloud Scripter & RLVR Recursive Training Pipeline



```

|   - Execution testing against mocked inputs
|   - Behavioral reward scoring (+1.0 for valid recovery, -1.0 fail)
| 4. Produces updated model weights / adapter checkpoints / prompt
packs.

```

Downstream Artifact Sync

Deployment Back to Edge & Local Tiers

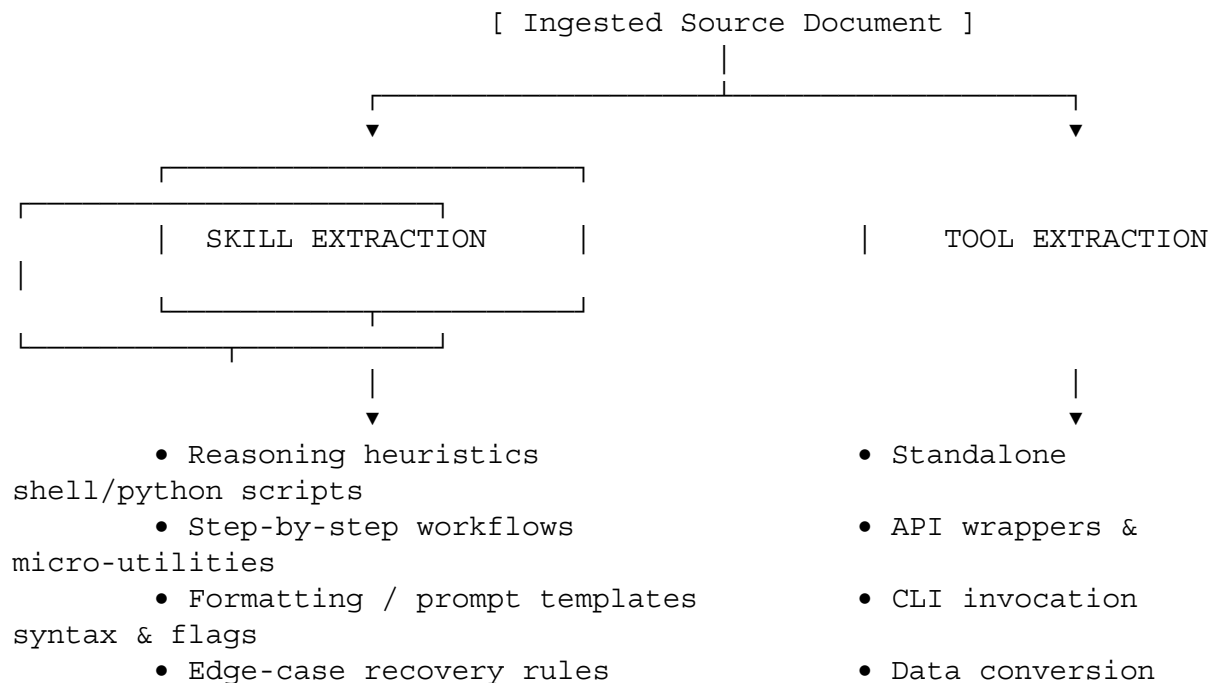
```

- Updated Prompt Skills    → 04_skills_runtime/prompt_skills/
- Updated Extracted Tools → 04_skills_runtime/extracted_tools/
- Verified Run Logs       → 05_episodic_logs/rlvr_verifiers/

```

6. Universal Skill & Tool Extraction Engine

The compilation agent must process all incoming documentation through a dual-channel extraction filter.



```

routines
      |
      ▼
[ 04_skills_runtime/      ]
]
      |
      ▼
[ prompt_skills/*.md      ]
]
      |
      ▼
[ extracted_tools/*
]

```

Extraction Classification Rules

1. **Targeting Skills:** If the passage describes *how to reason*, *how to structure an analysis*, or *how to coordinate a sequence of tasks*, format it as a markdown skill with YAML frontmatter into 04_skills_runtime/prompt_skills/.
2. **Targeting Tools:** If the passage contains *executable code*, *CLI command flags*, *API request formats*, or *data transformation utilities*, extract it into an executable script (.py or .sh) with explicit argument parsing into 04_skills_runtime/extracted_tools/.
3. **Registration:** Update the local 04_skills_runtime/manifest.jsonl immediately upon extraction.

7. Hierarchical manifest.jsonl Specification for Distributed RAG

Every directory root must hold a dedicated manifest.jsonl file. This prevents full-vault scanning during localized RAG operations.

JSONL Schema Structure

Each line in any manifest.jsonl must strictly adhere to this schema:

```

{
  "id": "UUID-OR-PATH-HASH",
  "path": "relative/path/to/file.ext",
  "tier": 4,
  "category": "extracted_tool",
  "sha256":
  "e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855",
  "tokens": 412,
  "semantic_summary": "CLI utility to extract audio streams from MP4
video containers.",
  "entities_extracted": ["ffmpeg", "audio_processing", "mp4_to_wav"],
  "skills_tools_extracted": [
    {"type": "tool", "name": "extract_audio_stream", "path":
  "04_skills_runtime/extracted_tools/cli/extract_audio.sh"},
    {"type": "skill", "name": "audio_preprocessing_policy", "path":
  "04_skills_runtime/prompt_skills/audio_prep.md"}
  ],
  "provenance_source":

```

```
"01_raw_sources/pdf/media_processing_guide.pdf",  
  "last_synced": "2026-09-03T17:05:00Z"  
}
```

RAG Query Traversal Protocol

1. **Directory-Level Scope Matching:** The RAG agent inspects root manifest.jsonl to identify candidate subdirectories.
2. **Localized Search:** The agent reads only the relevant sub-tier manifest.jsonl (e.g., 04_skills_runtime/manifest.jsonl), matching semantic summaries and entities.
3. **Targeted Ingestion:** Only the exact chunk or tool indicated by the manifest record is loaded into active Working Memory.

8. Master Agent Execution Lifecycle

When running an active session, the planning agent executes the following phased protocol:

[PHASE 1: INGEST & EXTRACT]

1. Detect new files in 01_raw_sources/.
2. Parse file content:
 - Distill core knowledge -> 02_master_wiki/
 - Extract procedural behaviors -> 04_skills_runtime/prompt_skills/
 - Extract executable utilities -> 04_skills_runtime/extracted_tools/
3. Generate RAG entries and update both local and root manifest.jsonl files.

[PHASE 2: EXECUTE & TELEMETRY]

1. Answer query or run user task using Tier 4 tools and Tier 3 fast cache.
2. Log all model contributions (Query, Executor, Frontier) into 05_episodic_logs/.

[PHASE 3: END-OF-DAY AUDIT & RLVR SYNC]

1. Trigger P2P connection to home nodes.
2. Compile cross-model tool and execution logs.
3. Route compiled candidate scripts through the Sandboxed Red Auditor.
4. If approved:
 - Push trajectory batch to the GCS bucket.
 - Trigger cloud scripiter for recursive RLVR evaluation.
5. If rejected:
 - Flag failure in 05_episodic_logs/red_audit_sandbox/.
 - Update error mitigation policies in 04_skills_runtime/policies/.